

Synchronization in Java

Synchronization is the mechanism Java uses to control concurrent access to shared mutable data.

When multiple threads work with the same object or variable, their operations can overlap. If those operations are not coordinated, the program may produce incorrect or unpredictable results.

1. Multithreading and Shared Data

Multithreading means that more than one thread can make progress within the same program.

Consider a simple counter:

```
class Counter {  
    int count = 0;  
  
    void increment() {  
        count++;  
    }  
}
```

Now suppose two threads call `increment()` 1,000 times each:

```
public class Main {  
  
    public static void main(String[] args)  
        throws InterruptedException {  
  
        Counter counter = new Counter();  
  
        Thread t1 = new Thread(() -> {  
            for (int i = 0; i < 1000; i++) {  
                counter.increment();  
            }  
        });  
    }  
}
```

```

    }
  });

  Thread t2 = new Thread(() -> {
    for (int i = 0; i < 1000; i++) {
      counter.increment();
    }
  });

  t1.start();
  t2.start();

  t1.join();
  t2.join();

  System.out.println(counter.count);
}
}

```

The expected result is:

```
2000
```

However, the actual result may be less than 2000.

The reason is that:

```
count++;
```

is not one indivisible operation. Conceptually, it performs three steps:

1. Read the current value of count
2. Add 1
3. Write the new value back

Two threads can overlap during these steps:

Thread-1 reads 10
Thread-2 reads 10
Thread-1 writes 11
Thread-2 writes 11

Two increments were attempted, but the value increased only once. This is called a **lost update**.

2. What Synchronization Does

Synchronization allows a thread to enter a protected section only after acquiring the required monitor lock.

```
class Counter {  
    private int count = 0;  
  
    synchronized void increment() {  
        count++;  
    }  
  
    int getCount() {  
        return count;  
    }  
}
```

If Thread-1 is executing `increment()` on a particular `Counter` object, Thread-2 cannot enter another synchronized instance method guarded by the same object until Thread-1 releases the monitor.

Using `synchronized` here prevents the read-modify-write steps of `count++` from overlapping.

3. Why Synchronization Is Needed

Synchronization mainly provides three things.

3.1 Protection of Shared Data

When multiple threads access the same mutable data, synchronization prevents conflicting operations from running at the same time.

Examples include:

- Updating a counter
- Modifying a bank balance
- Adding or removing elements from a shared collection
- Changing the state of a shared resource

3.2 Atomicity

An operation is atomic when other threads cannot observe or interfere with it halfway through.

```
synchronized void increment() {  
    count++;  
}
```

The complete `count++` operation is protected by one monitor.

This does not mean that every statement inside Java automatically becomes a hardware-level atomic instruction. It means that no other thread using the same monitor can enter the protected section until the current thread completes it.

3.3 Memory Visibility

Modern processors and compilers may keep values in registers, processor caches, or reordered instruction sequences.

Without proper synchronization, one thread may update a variable while another thread continues observing an older value.

Monitor synchronization establishes a **happens-before relationship**:

```
A thread releases a monitor  
→ another thread later acquires the same monitor
```

→ the acquiring thread can observe the writes made before the release

Therefore, synchronization provides both:

Mutual exclusion
+
Memory visibility

4. Synchronization Does Not Stop All Threads

Synchronization does not pause the entire application.

It only controls access to the code protected by a particular monitor.

If Thread-1 holds the monitor of an object:

- Thread-2 cannot enter code protected by the same monitor.
- Thread-2 can still execute unrelated code.
- Thread-2 can enter code protected by a different monitor.
- Other threads can continue running normally.

The correct mental model is:

Synchronization restricts access to a protected region; it does not stop multithreading.

Object Lock and Monitor Lock

5. What Is a Monitor Lock?

Every Java object can be used as a monitor.

A monitor acts like a permission mechanism:

A thread must acquire the object's monitor before entering code synchronized on that object.

The monitor is managed by the JVM. It is not a normal field that appears inside the Java class.

```
Object lock = new Object();
```

The `lock` object can be used as a monitor:

```
synchronized (lock) {  
    // protected code  
}
```

6. How Monitor Acquisition Works

When a thread reaches synchronized code:

1. It attempts to acquire the required monitor.
2. If the monitor is available, the thread acquires it and enters.
3. If another thread owns the monitor, the current thread cannot enter.
4. The current thread remains blocked until the monitor becomes available.
5. The monitor is released when the owning thread exits the synchronized region.

Example:

```
class Test {  
  
    synchronized void show() {  
        System.out.println(  
            Thread.currentThread().getName() + " entered"  
        );  
  
        try {  
            Thread.sleep(1000);  
        } catch (InterruptedException e) {  
            Thread.currentThread().interrupt();  
        }  
    }  
}
```

```

        System.out.println(
            Thread.currentThread().getName() + " exiting"
        );
    }
}

```

```

public class Main {

    public static void main(String[] args) {
        Test test = new Test();

        Thread t1 = new Thread(test::show, "Thread-1");
        Thread t2 = new Thread(test::show, "Thread-2");

        t1.start();
        t2.start();
    }
}

```

Both threads call the same synchronized method on the same object. Therefore, only one thread can execute `show()` at a time.

7. The Lock Belongs to the Object, Not the Method

A synchronized method does not have its own independent lock.

For an instance method:

```

synchronized void show() {
    // code
}

```

the monitor belongs to the current object:

```

this

```

The method is conceptually equivalent to:

```
void show() {  
    synchronized (this) {  
        // code  
    }  
}
```

This distinction is important because multiple synchronized methods on the same object use the same monitor.

8. Multiple Synchronized Methods on the Same Object

```
class Test {  
  
    synchronized void m1() {  
        System.out.println("m1");  
    }  
  
    synchronized void m2() {  
        System.out.println("m2");  
    }  
}
```

Suppose Thread-1 enters `m1()` on a particular `Test` object.

Thread-2 cannot enter `m2()` on that same object because both methods require the monitor of the same object.

```
Thread-1 → test.m1()  
Thread-2 → test.m2()
```

```
Same object  
→ same monitor  
→ mutual blocking
```

The methods are different, but the monitor is the same.

9. Different Objects Use Different Monitors

```
Test first = new Test();  
Test second = new Test();
```

Now suppose:

```
Thread-1 → first.m1()  
Thread-2 → second.m1()
```

Both threads may execute simultaneously because the two objects have different monitors.

```
first object → first monitor  
second object → second monitor
```

Therefore:

Synchronization depends on the monitor object being shared, not merely on the method or class containing the code.

Synchronized Blocks

10. Why Use a Synchronized Block?

A synchronized instance method protects the entire method.

```
synchronized void increment() {  
    // entire method is protected  
}
```

Sometimes only a small part of the method accesses shared mutable data. Protecting the whole method may keep the monitor occupied longer than

necessary.

A synchronized block allows the program to protect only the critical section.

```
synchronized (monitorObject) {  
    // critical section  
}
```

11. Example of a Synchronized Block

```
class Counter {  
    private int count = 0;  
  
    void increment() {  
        System.out.println("Before increment");  
  
        synchronized (this) {  
            count++;  
        }  
  
        System.out.println("After increment");  
    }  
}
```

Only the code inside:

```
synchronized (this) {  
    count++;  
}
```

requires the monitor of the current `Counter` object.

The code before and after the block does not require that monitor.

This can reduce unnecessary contention, provided the shared state is still protected correctly.

12. Critical Section

A **critical section** is the part of the program that accesses shared mutable state and must not be executed concurrently by threads using the same synchronization policy.

```
synchronized (this) {  
    count++;  
}
```

The goal is not to make synchronized blocks as small as possible at any cost.

The complete operation that must remain consistent should be protected under the same monitor.

For example:

```
synchronized (this) {  
    if (balance >= amount) {  
        balance = balance - amount;  
    }  
}
```

The condition check and the update must remain inside the same synchronized section.

Custom Lock Objects

13. Using a Dedicated Lock Object

A class does not have to synchronize on `this`.

It can use a dedicated private object:

```
class Counter {  
    private final Object lock = new Object();  
    private int count = 0;  
}
```

```
void increment() {  
    synchronized (lock) {  
        count++;  
    }  
}
```

Here, the critical section is protected by the monitor of `lock`, not by the monitor of the `Counter` instance itself.

A dedicated lock object gives the class more control over which operations block one another.

It should normally be:

```
private final
```

This prevents external code from acquiring the same monitor accidentally and prevents the lock reference from being replaced.

14. Multiple Locks Inside One Object

A class may use different monitors for independent state:

```
class AccountManager {  
    private final Object accountLock = new Object();  
    private final Object auditLock = new Object();  
  
    private int balance;  
    private int auditEntries;  
  
    void updateBalance(int amount) {  
        synchronized (accountLock) {  
            balance += amount;  
        }  
    }  
}
```

```
void recordAuditEntry() {
    synchronized (auditLock) {
        auditEntries++;
    }
}
```

The two methods may run concurrently because they use different monitors and protect independent state.

```
updateBalance()    → accountLock
recordAuditEntry() → auditLock
```

15. Different Locks Are Safe Only for Independent State

Consider this design:

```
class Bank {
    private final Object depositLock = new Object();
    private final Object withdrawLock = new Object();

    private int balance;

    void deposit(int amount) {
        synchronized (depositLock) {
            balance += amount;
        }
    }

    void withdraw(int amount) {
        synchronized (withdrawLock) {
            balance -= amount;
        }
    }
}
```

```
}  
}
```

Although `deposit()` and `withdraw()` use synchronized blocks, they modify the same `balance` using different monitors.

Therefore, they can still execute concurrently and corrupt the shared state.

To protect the same invariant, use the same monitor:

```
class Bank {  
    private final Object balanceLock = new Object();  
    private int balance;  
  
    void deposit(int amount) {  
        synchronized (balanceLock) {  
            balance += amount;  
        }  
    }  
  
    void withdraw(int amount) {  
        synchronized (balanceLock) {  
            balance -= amount;  
        }  
    }  
}
```

The rule is:

Data that participates in the same consistency rule must be protected by the same lock.

16. Synchronization Depends on the Same Monitor

Two pieces of identical code are not synchronized with each other merely because they look the same.

They coordinate only when they acquire the same monitor object.

```
synchronized (lock) {  
    count++;  
}
```

All threads must use the same `lock` reference for this protection to work.

17. Common Mistake: Creating a New Lock Every Time

Incorrect:

```
synchronized (new Object()) {  
    count++;  
}
```

Every execution creates a new object:

```
Call 1 → monitor A  
Call 2 → monitor B  
Call 3 → monitor C
```

Because the threads acquire different monitors, they do not block one another.

Correct:

```
class Counter {  
    private final Object lock = new Object();  
    private int count;  
  
    void increment() {  
        synchronized (lock) {  
            count++;  
        }  
    }  
}
```

Now every call uses the same monitor.

Static Synchronization

18. Why Static Synchronization Is Different

Instance fields belong to individual objects:

```
class Counter {  
    int count;  
}
```

Static fields belong to the class:

```
class Counter {  
    static int count;  
}
```

A synchronized instance method uses the current object's monitor. That is not enough to coordinate access to static state across multiple objects.

For static synchronized methods, Java uses the monitor of the corresponding `Class` object.

19. Static Synchronized Method

```
class Counter {  
    private static int count = 0;  
  
    static synchronized void increment() {  
        count++;  
    }  
}
```

This is conceptually equivalent to:

```
class Counter {
    private static int count = 0;

    static void increment() {
        synchronized (Counter.class) {
            count++;
        }
    }
}
```

The monitor is:

`Counter.class`

It is not:

`this`

A static method has no `this` reference.

20. Multiple Objects Still Share the Class Monitor

```
Counter first = new Counter();
Counter second = new Counter();
```

Static methods should normally be called through the class name:

```
Thread t1 = new Thread(Counter::increment);
Thread t2 = new Thread(Counter::increment);
```

Both calls require the same monitor:

`Counter.class`

Therefore, they cannot execute the static synchronized method simultaneously.
The number of `Counter` objects does not create additional class monitors.

21. Object Monitor vs Class Monitor

```
class Test {  
  
    synchronized void instanceMethod() {  
        // monitor: this  
    }  
  
    static synchronized void staticMethod() {  
        // monitor: Test.class  
    }  
}
```

Suppose:

```
Thread-1 → test.instanceMethod()  
Thread-2 → Test.staticMethod()
```

These methods can execute concurrently because they use different monitors:

```
instanceMethod() → test object  
staticMethod()   → Test.class object
```

Therefore:

A synchronized instance method and a synchronized static method do not automatically block one another.

They block only if both are explicitly synchronized on the same monitor.

Conceptual Monitor Internals

22. A Simplified Mental Model

Conceptually, a monitor must track information such as:

```
Current owning thread
Re-entry count
Threads waiting to acquire the monitor
Threads waiting after calling wait()
```

A simplified model might look like:

```
Monitor {
    owner
    holdCount
    entrySet
    waitSet
}
```

This is a teaching model, not the exact JVM memory layout.

The JVM may use optimized representations such as lightweight locking, lock inflation, operating-system primitives, or other implementation-specific mechanisms.

23. Entering a Synchronized Region

When a thread reaches:

```
synchronized (lock) {
    // critical section
}
```

conceptually:

1. The JVM checks whether the thread can acquire lock's monitor.
2. If the monitor is available, the thread becomes its owner.
3. If another thread owns it, the current thread cannot enter.
4. The blocked thread becomes eligible again when the monitor is released.

Acquiring the monitor is performed atomically by the JVM.

Two threads cannot both successfully become the first owner of the same monitor at the same time.

The exact implementation is JVM-dependent and should not be reduced to a guarantee that one particular CAS instruction is always used.

24. Exiting a Synchronized Region

The monitor is released when execution leaves the synchronized method or block.

This applies during normal completion:

```
synchronized (lock) {  
    // work  
}
```

It also applies when the code exits because of an exception:

```
synchronized (lock) {  
    throw new RuntimeException();  
}
```

The JVM still releases the monitor while unwinding the synchronized region.

This automatic release is one major advantage of `synchronized`.

25. Reentrant Nature of `synchronized`

Java monitors are reentrant.

A thread that already owns a monitor may acquire the same monitor again.

```
class Service {  
  
    synchronized void methodA() {  
        methodB();  
    }  
}
```

```
synchronized void methodB() {
    System.out.println("Inside methodB");
}
```

When a thread enters `methodA()`, it acquires the monitor of `this`.

It can then call `methodB()` on the same object without blocking itself because it already owns that monitor.

The JVM internally tracks the number of times the thread has acquired the monitor. The monitor is fully released only after the matching synchronized entries are exited.

Common Synchronization Mistakes

26. Synchronizing on Different Objects

```
synchronized (lock1) {
    count++;
}
```

and:

```
synchronized (lock2) {
    count++;
}
```

do not protect `count` from each other when `lock1` and `lock2` refer to different objects.

Shared state must follow one consistent locking policy.

27. Replacing the Lock Reference

Avoid:

```
class Counter {
    private Object lock = new Object();

    void changeLock() {
        lock = new Object();
    }
}
```

Threads may begin synchronizing on different monitor objects.

Prefer:

```
private final Object lock = new Object();
```

28. Exposing the Lock Publicly

Avoid using a publicly accessible lock:

```
public final Object lock = new Object();
```

External code could synchronize on it and unexpectedly block the class.

Prefer a private lock:

```
private final Object lock = new Object();
```

29. Protecting Only Part of a Compound Operation

Incorrect:

```
if (balance >= amount) {
    synchronized (lock) {
        balance -= amount;
    }
}
```

```
}  
}
```

The condition is checked outside the monitor. Another thread may change `balance` before the synchronized update begins.

Correct:

```
synchronized (lock) {  
    if (balance >= amount) {  
        balance -= amount;  
    }  
}
```

The check and update form one atomic operation with respect to the same monitor.

Summary

Synchronization is used to coordinate threads that access shared mutable state.

```
synchronized instance method  
→ acquires the monitor of this  
  
synchronized static method  
→ acquires the monitor of ClassName.class  
  
synchronized block  
→ acquires the monitor of the object written in parentheses
```

The most important rules are:

1. Synchronization works only when competing threads use the same monitor.
2. A lock belongs to an object, not to a method.
3. Different objects have different monitors.
4. Static synchronized methods use the class object's monitor.

5. The entire consistency rule must be protected under one monitor.
 6. Synchronization provides mutual exclusion and memory visibility.
 7. A monitor is automatically released when the synchronized region ends.
 8. Java monitors are reentrant.
 9. Dedicated lock objects should normally be `private final`.
 10. Using different locks for the same shared state does not provide safety.
-

Next Topic: Inter-Thread Communication

Synchronization prevents multiple threads from entering the same protected region at the same time.

However, synchronization alone does not tell a thread what to do when it acquires the monitor but the required condition is not yet satisfied.

For example:

```
Consumer acquires the monitor, but no item is available.  
Producer acquires the monitor, but the shared buffer is already full.
```

Continuously checking the condition wastes CPU and may prevent other threads from changing the state.

This leads to the next topic:

```
wait();  
notify();  
notifyAll();
```

These methods allow synchronized threads to wait for a condition and notify one another when the shared state changes.